

Documento de análisis taller 2

Integrates:

- Juan David Bolaños Fernandez – 202514487
- José David Huertas Tapiero - 202512860
- David Orlando Bastos Castillejo - 202520216
- Ivan David Vargas Reyes – 202524636

Resultados Obtenidos:

Punto 1

Para el caso del punto 1, se realizó la implementación del algoritmo Ascenso de Colina. Este algoritmo es consistente entre cada ejecución, retornando el mismo resultado al no tener factores estocásticos. El algoritmo suele finalizar pronto debido a que encuentra una desmejora de manera muy pronta, y en este caso termina y retorna el último estado explorado por su propiedad de solo admitir las mejoras. Debido a esto el algoritmo suele finalizar muy pronto, llegando a finalizar en tan solo una iteración en muchos layouts o hasta 4 iteraciones en algún layout extenso como barrier_districts por ejemplo. Finalmente, este algoritmo retorna máximos locales debido a su propiedad, pero no es óptimo al momento de buscar un máximo global y realizar una optimización completa.

Punto 2

Para el caso del punto 2, se realizó la implementación del algoritmo Recocido Simulado. Este algoritmo ya implementa un factor estocástico, pues descarta un estado por medio de la probabilidad en caso de que no sea mejor que el estado actual. Para este caso, el algoritmo se extenderá hasta el límite de iteraciones, el cual llega a ocurrir antes que el parámetro de temperatura mínima. Además, el algoritmo encuentra en todos los casos un muy posible máximo global. Por otra parte, el algoritmo sí se ve afectado por el cambio en la configuración a comparación de Ascenso en Colina, esto debido a que es mucho más probable que el algoritmo alcance el límite de iteraciones y además se agrega un cambio en el factor de cambio de temperatura.

Punto 3 Algoritmo Genetico

Efecto de inicialización, aleatoriedad y parámetros principales en algoritmo genético.

Parámetro	Configuración 1: Explotación Conservadora (Explotación)	Configuración 2: Exploración Diversificada (Exploración)	Configuración 3: Mayor Exploración
Population_size	30	100	200
Generations	100	50	50
Mutation_probability	0.02	0.20	0.30
Elite_size	4	1	1

Resultados:

Risk_Corridor

Configuracion	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Configuracion 1	116	116	0	4000	0.159
Configuracion 2	116	116	0	4000	0.163
Configuracion 3	116	116	0	4000	0.142

Balanced City

Configuracion	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Configuracion 1	281	281	0	4000	0.189
Configuracion 2	281	281	0	4000	0.219
Configuracion 3	281	281	0	4000	0.168

Análisis en punto 6a.b

Punto 4. Búsqueda Adversaria: Minimax.

a. Componentes de la formulación entregada:

Estado Inicial (S_0)	La cuadrícula con la posición inicial del defensor (MAX), la posición inicial del intruso (MIN) y todas las terminales críticas pendientes (desactivadas). El primer turno siempre correspondiente al defensor, MAX.
Turno (s)	Identificador que alterna entre el agente defensor (0) y el agente intruso (1) respetando el orden fijo de primero MAX, luego MIN en cada nivel del árbol.
Acciones (s)	Para el agente en turno, realizar cualquier movimiento legal (arriba, abajo, izquierda, derecha) o permanecer quieto, excluyendo siempre aquellos movimientos que hagan al agente atravesar bloqueos o salir de los límites del mapa.
Resultado (s,a)	Dado un estado y una acción legal, se produce un nuevo estado en donde se actualiza la posición del agente en turno o se activa una terminal crítica solo si el defensor está parado sobre esta, y se cede el turno al otro agente.
EsTerminal (s)	El estado es terminal únicamente si el agente defensor ha activado todas las terminales críticas (victoria del defensor), el agente intruso ha interceptado al agente defensor (derrota del defensor), o se ha llegado al límite de profundidad, es decir, se han terminado las rondas y ningún agente ganó.
Utilidad (s,j)	Dependiendo del estado terminal de la partida puede ser: +1000 en caso de la victoria del agente defensor, -1000 en caso de la derrota del agente defensor, una aproximación mediante una función heurística en caso de llegar al límite de profundidad sin un ganador fijo.

d. Comparación de profundidades entre layouts:

- Profundidad 2 plies:

layout	Acción inicial	Resultado	Nodos explorados	Nodos Intruso
single_terminal	East	Victoria	112	73
deceptive_distance	West	Límite	639	551

- Profundidad 4 plies:

layout	Acción inicial	Resultado	Nodos explorados	Nodos Intruso
single_terminal	East	Victoria	1153	366
deceptive_distance	West	Victoria	4412	2005

Al aumentar la profundidad, cada agente es capaz de prever más posibles acciones que tomará su contrincante, por esto en algunos mapas se obtienen resultados diferentes. Propiamente, en el caso de deceptive_distance, al tener 4 plies, el agente

defensor (MAX) logra prever que si en una ronda en particular se va hacia la izquierda quedará encerrado por el intruso (MIN); esto no pasa cuando se tienen únicamente 2 plies y la partida llega al límite de rondas ya que el defensor si queda encerrado por el intruso.

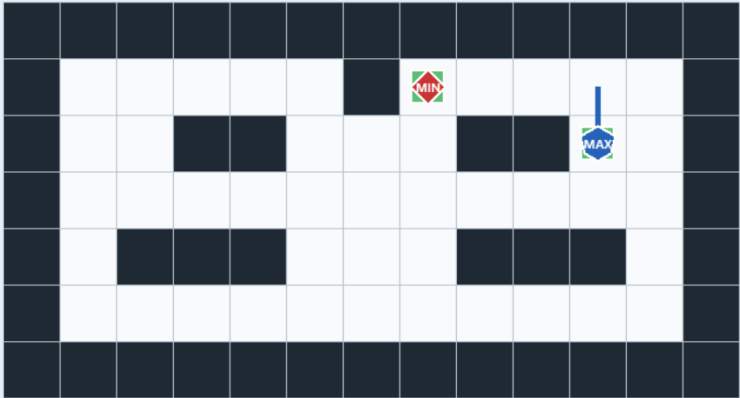
Punto 5.

Apendice c)

Deseamos comprobar que para una misma comparación tanto alfa beta como minimax generan el mismo resultado, y revisar como varían los nodos explorados.

Análisis en el mapa "dual_defense":

Resultado de AlphaBeta:



Mapa de juego 12x12 con obstáculos (celdas oscuras) y agentes MAX (azul) y MIN (rojo). El agente MAX está en la celda (7, 8) y el agente MIN está en la celda (6, 7).

RED CENTINELA

EJECUCIÓN
AlphaBetaAgent
Defensor MAX: South
Fotograma 12/12 · Acción inicial: East

MÉTRICAS DEL FOTOGAMA

Profundidad	4 plies
Rondas	5
Terminales pendientes	0
Puntaje	1000.0
Nodos MAX	179
Nodos MIN	253

DETALLE DEL PASO
RESULTADO FINAL
Resultado: victoria
Finalización: activación completa

LEYENDA
MAX Defensor MAX
MIN Intruso MIN
T Terminal pendiente
OK Terminal activada por MAX
→ Último movimiento

ESTADO ACTUAL DE LA MISIÓN

ÚLTIMO TURNO MAX agente	RONDAS 5 completas	PENDIENTES 0 terminales	PUNTAJE 1000.0 defensor
--------------------------------------	---------------------------------	--------------------------------------	--------------------------------------

Progreso de la ejecución: 11/11

 Pausado

Resultado minimax:

RED CENTINELA

EJECUCIÓN

MinimaxAgent

Defensor MAX: South

Fotograma 12/12 · Acción inicial: East

MÉTRICAS DEL FOTOGRAMA

Profundidad	4 plies
Rondas	5
Terminales pendientes	0
Puntaje	1000.0
Nodos MAX	1024
Nodos MIN	253

DETALLE DEL PASO

RESULTADO FINAL
Resultado: victoria
Finalización: activación completa

LEYENDA

- MAX: Defensor MAX
- MIN: Intruso MIN
- T: Terminal pendiente
- OK: Terminal activada por MAX
- : Último movimiento

ESTADO ACTUAL DE LA MISIÓN

ÚLTIMO TURNO MAX agente	RONDAS 5 completas	PENDIENTES 0 terminales	PUNTAJE 1000.0 defensor
--	---	--	--

Progreso de la ejecución: 11/11

↺
⏮
▶
⏭
Pausado

Como se puede observar, ambos agentes tomaron exactamente la misma decisión “East” (ver en la foto de la terminal), y ambos juegos resultaron en el mismo estado terminal.

Ahora bien, como se observa en la siguiente foto:

```

Ejecutando:
C:\Users\davob\AppData\Local\Programs\Python\Python310\python.exe C:\Users\davob\OneDrive\Desktop\IA\Taller2-IA\main.py -m adversarial -a AlphaBetaAgent -l dual_terminal -x 0.55 -d 4 --max-rounds 40

AlphaBetaAgent | profundidad=4 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=179 | nodos_intruso=253 | tiempo=0.010636s

Ejecución finalizada correctamente.
Presiona Enter para volver al menú...

Ejecutando:
C:\Users\davob\AppData\Local\Programs\Python\Python310\python.exe C:\Users\davob\OneDrive\Desktop\IA\Taller2-IA\main.py -m adversarial -a MinimaxAgent -l dual_terminal -x 0.55 -d 4 --max-rounds 40

MinimaxAgent | profundidad=4 plies | acción_inicial=East | resultado=victoria | finalización=activación completa | nodos=1024 | nodos_intruso=253 | tiempo=0.021840s

```

La decisión que a minimax le costó 1024 nodos deducir, a al agente AlphaBeta le costó únicamente 179 nodos. Esto repercutió en el tiempo de ejecución, pues el análisis que a minimax le tomó 0.021 segundos, le tomó únicamente 0.010 segundos al agente AlphaBeta.

Análisis en el mapa “Medium_defense”:

Resultado AlphaBeta:

Como se puede observar, en ambos casos los agentes decidieron que MAX debe moverse hacia el "north" (observar en la foto de la terminal). Además, ambos agentes llegaron al mismo estado terminal y mismo puntaje.

Ahora bien, como se observa en el resultado de las ejecuciones:

```
Ejecutando:
C:\Users\davob\AppData\Local\Programs\Python\Python310\python.exe C:\Users\davob\OneDrive\Desktop\IA\Taller2-IA\main.py -m adversarial -a MinimaxAgent -l medium_defense
-x 0.55 -d 4 --max-rounds 40

MinimaxAgent | profundidad=4 plies | acción_inicial=North | resultado=límite | finalización=límite de rondas | nodos=8998 | nodos_intruso=4463 | tiempo=0.138552s

Ejecución finalizada correctamente.
Presiona Enter para volver al menú...clear

Ejecutando:
C:\Users\davob\AppData\Local\Programs\Python\Python310\python.exe C:\Users\davob\OneDrive\Desktop\IA\Taller2-IA\main.py -m adversarial -a AlphaBetaAgent -l medium_defens
e -x 0.55 -d 4 --max-rounds 40

AlphaBetaAgent | profundidad=4 plies | acción_inicial=North | resultado=límite | finalización=límite de rondas | nodos=2223 | nodos_intruso=4463 | tiempo=0.117462s

Ejecución finalizada correctamente.
Presiona Enter para volver al menú...|
```

La decisión que AlphaBeta dedujo en 2223 nodos, le costó a minimax la exploración de 8998 nodos. Nuevamente, la diferencia en tiempo apoya la idea de que AlphaBeta es más eficiente, pues minimax se tomó 0.13 segundos, mientras que AlphaBeta se tomó 0.11 segundos.

Conclusión apéndice c:

En conclusión, el agente AlphaBeta es capaz de llegar exactamente al mismo resultado que al agente MiniMax, la diferencia es que recorta múltiples nodos que no influyen en el resultado, reduciendo drásticamente la cantidad de nodos explorados, y por ende la cantidad de tiempo y recursos utilizados para resolver el problema. Estos resultados experimentales coinciden perfectamente con la teoría del curso, pues desde un principio se nos indicó que AlphaBeta no pretende cambiar el resultado otorgado por MiniMax, sino que busca reducir el trabajo realizado por el agente para llegar a este.

Apéndice d)

En la función de evaluación buscamos asignarle un valor a cada estado no terminal (en los estados terminales ya hay una evaluación dada entonces no aplica), revisando que tan beneficioso es ese estado del juego para el defensor (MAX). Para esto se tienen en cuenta los siguientes parametros del estado:

- Score. Este parámetro representa las recompensas y penalizaciones acumuladas hasta x estado. En este caso, un mayor score implica directamente un mejor desempeño de MAX. Por este motivo, no se modifica el valor y se usa como punto de partida para la evaluación.
- Distancia defensor-intruso. Este parámetro representa que tan lejos está el defensor del intruso (MIN de MAX). A mayor distancia se considera que el defensor está más

“seguro”. Dado que la distancia es proporcional a la mejora tampoco se modifica el valor con pesos.

- Distancia al terminal pendiente más cercano. Este parámetro representa que tan lejos está el defensor de la terminal no reparada más cercana. Este parámetro evalúa y penaliza un estado cuando el defensor está lejos de la próxima terminal. Ahora bien, note que a menor distancia mejor desempeño de MAX. Pero como la función de evaluación se planteó para maximizarse (mayor valor mejor desempeño) no podemos sumar directamente la distancia, pues menor distancia daría un número menor que una distancia mayor en la función. Por este motivo, se modifica este parámetro invirtiéndolo y sumándolo de la siguiente manera:
 $100/(1+\text{distanciaMasCercano})$

De esta forma, un número alto de distancia reducirá el índice de 100, mientras que una distancia pequeña lo reducirá menos, manteniendo la función de evaluación coherente.

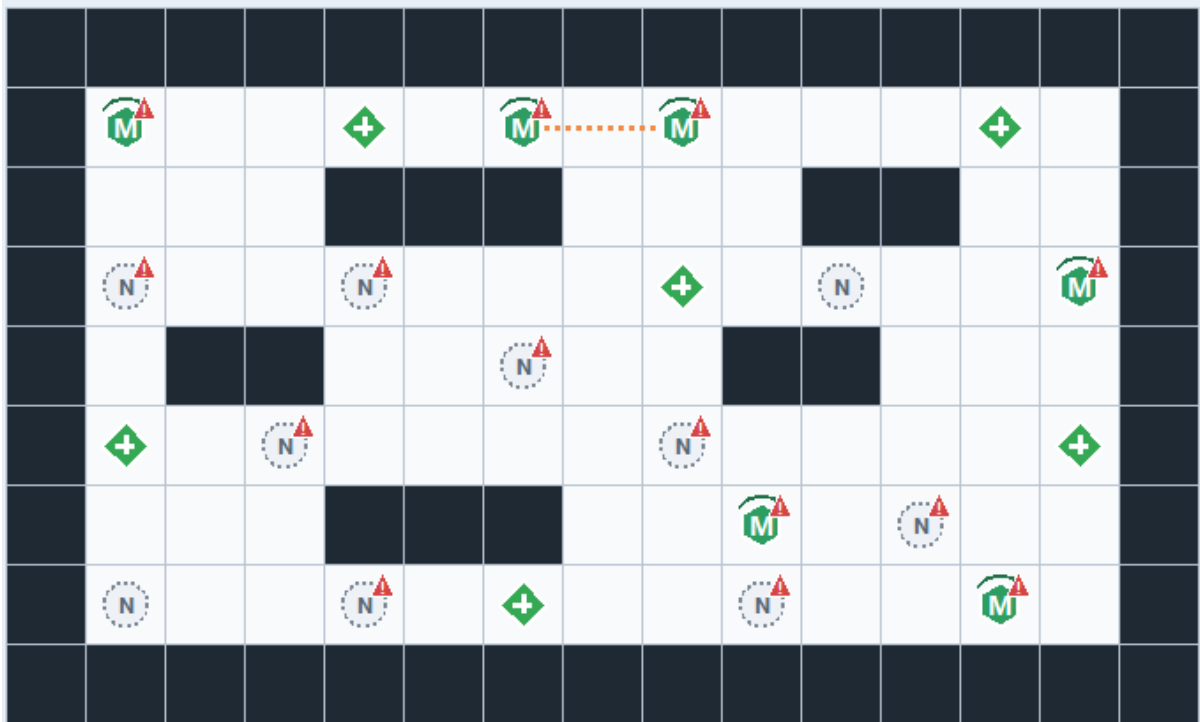
-terminales pendientes. Este parámetro busca penalizar los estados donde a MAX le faltan muchas terminales por revisar. Se utiliza el peso 10 para agregar relevancia a la penalización, pues note que el número de terminales faltantes en los ejercicios de prueba varía entre 0 y 5 por lo que restar el número directamente produciría una penalización poco relevante a comparación de la magnitud de decenas en el score y demás parámetros. Por este motivo, y para igualar la escala de decenas, se multiplica este número por el peso 10.

En conjunto, todos estos parámetros permiten notar que un estado favorable para el defensor tendrá un buen score, una distancia segura respecto al intruso, una distancia pequeña hacia el próximo terminal pendiente, y un número reducido de terminales pendientes. Estos parámetros también permiten a los agentes navegar estados intermedios y escoger aquellos que prometan mayor progreso.

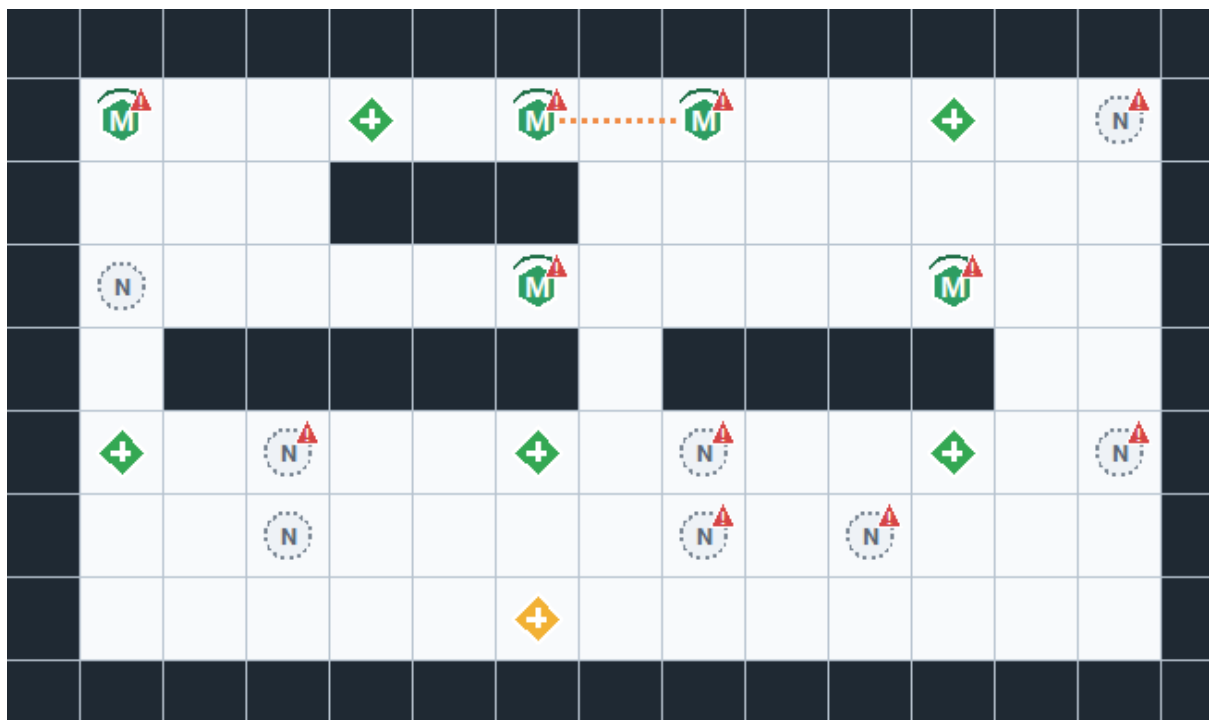
Punto 6a. Análisis de Algoritmos y resultados

a. ¿Qué características de los layouts explican las diferencias observadas entre ascenso de colina, recocido simulado y algoritmo genético?

Balanced City (Layout con distribución uniforme de nodos críticos+ y sitios con riesgo operativo alto-)



Risk Corridor (Layout con separacion marcada de nodos criticos+ y sitios con riesgo operativo alto-)



Las diferencias de desempeño entre los tres algoritmos se explican principalmente por tres características estructurales de los layouts: la disposición de las barreras (que fragmentan el espacio de búsqueda y alteran las distancias reales usadas en `configuration_score`), la distribución espacial de los nodos críticos (que determina dónde se concentra el beneficio de

cobertura) y la concentración de sitios con riesgo operativo alto (que penaliza regiones completas del mapa).

Ascenso de colina explora exclusivamente el vecindario inmediato de la configuración actual, por lo que su desempeño depende casi por completo del punto de partida. Cuando la inicialización cae en una zona dominada por sitios de alta exposición, el algoritmo puede alcanzar rápidamente un máximo local sin mejora estricta disponible, deteniéndose ahí aunque existan regiones con mejor puntaje en otra parte del mapa. De forma similar, cuando los vecinos en una posición ofrecen una cobertura y redundancia casi idénticas, se generan mesetas que también detienen el algoritmo por ausencia de mejora estricta. En ambos casos, las barreras agravan el problema al reducir el número de vecinos alcanzables y al aumentar la distancia real hacia sitios potencialmente mejores.

Recocido simulado mitiga estas limitaciones gracias a la aceptación probabilística de movimientos que empeoran momentáneamente el puntaje. Con una temperatura inicial suficientemente alta, el algoritmo puede "atravesar" configuraciones intermedias desfavorables, es decir zonas de alta exposición o mesetas que *Ascenso de colina* jamás abandonaría, aumentando la probabilidad de alcanzar el máximo global independientemente del punto de partida. Por ejemplo, en un layout como *risk_corridor*, donde el óptimo global está separado del resto del mapa por una barrera, *Recocido simulado* con temperatura alta logra cruzar esa franja con mayor frecuencia que *Ascenso de colina*, que queda atrapado antes de alcanzarla. Sin embargo, este comportamiento depende fuertemente del programa de enfriamiento: temperaturas iniciales bajas o factores de enfriamiento muy agresivos (α bajo) hacen que el algoritmo pierda esta capacidad de escape y se comporte de forma casi idéntica a *Ascenso de colina*.

El *algoritmo genético*, al operar sobre una población completa en lugar de una única trayectoria, distribuye naturalmente sus individuos entre distintas regiones del mapa, lo cual le da una ventaja estructural en layouts fragmentados. El cruce de un punto permite combinar segmentos de individuos ubicados en clústeres distintos, generando descendencia que redistribuye módulos entre regiones de forma que ningún movimiento local de *Ascenso de colina* o *Recocido simulado* podría producir en un solo paso. La selección por torneo y el elitismo concentran progresivamente la población en las zonas de mayor aptitud, ampliando así la exploración inicial del espacio de búsqueda antes de converger. No obstante, esta ventaja no garantiza encontrar el máximo global: si la población inicial o la mutación no mantienen suficiente diversidad, el algoritmo puede converger rápidamente hacia una zona mejor que parece buena de acuerdo con las primeras generaciones, pero deja sin explorar otras regiones del mapa con potencial igual o superior.

En síntesis, cuanto más fragmentado esté el layout por barreras y cuanto más desigual sea la distribución de exposición y de nodos críticos, mayor es la ventaja de *Recocido simulado* y del *Algoritmo genético* sobre *Ascenso de colina*, ya que ambos incorporan mecanismos para escapar de óptimos locales dados por el layout de un mapa.

b. ¿Qué efecto tuvieron la inicialización, la aleatoriedad y los parámetros principales sobre los resultados de los algoritmos de optimización?

Algoritmo	Descripción
Ascenso en Colina	La inicialización en este algoritmo es demasiado importante, pues es el factor principal que determina el resultado ya que el algoritmo obtendrá el óptimo local más cercano al estado inicial. Dicho esto, según cual sea nuestro punto de partida vamos a obtener un resultado u otro. En este algoritmo no hay aleatoriedad, pues iniciando en el mismo estado siempre se llegará al mismo resultado y no hay algún factor estocástico que lo afecte. El parámetro principal "max_iterations" no es tan relevante en este caso, pues es demasiado improbable llegar a un límite de iteraciones mientras este sea generoso debido a que el algoritmo suele encontrar un óptimo local muy rápido.
Recorrido Simulado	La inicialización en este algoritmo puede influir en el desarrollo de este, pero no es tan determinante como en Ascenso de Colina, pues este puede alejarse del estado inicial a comparación. En este algoritmo hay dos factores de aleatoriedad, en los que el algoritmo puede tener un recorrido distinto a pesar de iniciar en el mismo estado inicial. Estos son la elección aleatoria del vecino candidato y la probabilidad de aceptar un movimiento igual o peor. Este algoritmo depende enormemente de los parámetros principales, los cuales son la temperatura inicial, la tasa de enfriamiento, el límite de iteraciones y la temperatura mínima necesaria para seguir ejecutando. El cambio de estos parámetros puede hacer que el algoritmo se detenga más rápido y

	por ello afecta el resultado al perder una parte del recorrido en la cual se pudo haber alcanzado un candidato a máximo global.
Algoritmo genetico	De acuerdo con la tabla en el punto 3, las configuraciones enfocadas en exploración (amplias poblaciones, alta probabilidad de mutación y bajo elitismo) tienden a tardarse menos que las configuraciones de explotación. Por otra parte, los algoritmos de explotación (Alto elitismo y mayor número de generaciones para sacar provecho de la convergencia) comparten el resultado obtenido, desviación y numero de evaluaciones con las configuraciones de alta explotación. La mayor diferencia visible fue el tiempo de la ejecución del algoritmo.

c. ¿Qué compromiso observa entre calidad de la solución y costo computacional en los algoritmos de optimización?

(Para las siguientes tablas se usaron los parámetros de los algoritmos por defecto)

Balanced city

Algoritmo	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Ascenso en Colina	278	278	0	181	0.006
Recorrido Simulado	283	283	0	751	0.044
Algoritmo genetico	281	281	0	4000	0.189

Risk Corridor

Algoritmo	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Ascenso en Colina	108	108	0	121	0.003
Recorrido Simulado	109	106	0	751	0.034
Algoritmo genetico	116	116	0	4000	0.159

Exposure_Tradeoff

Algoritmo	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Ascenso en Colina	271	271	0	289	0.013

Recorrido Simulado	271	271	0	751	0.047
Algoritmo genetico	271	271	0	4000	0.205

Metropolitan_Network

Algoritmo	Mejor	Promedio	Desviacion	Eval. Promedio	Tiemp. Promedio
Ascenso en Colina	569	569	0	271	0.016
Recorrido Simulado	580	580	0	751	0.064
Algoritmo genetico	580	580	0	4000	0.255

En cuanto a calidad de solución cada algoritmo dio el siguiente número de maximos globales:

Ascenso en colina: $\frac{1}{4}$

Recorrido Simulado: $\frac{3}{4}$

Algoritmo genético: $\frac{3}{4}$

El compromiso entre calidad y costo computacional se observa de forma distinta en cada algoritmo, y depende en gran medida de las características del layout.

Ascenso de colina es, con claridad, el más económico computacionalmente: entrega resultados de forma casi inmediata y con el menor número de evaluaciones de los tres algoritmos. Sin embargo, su calidad depende casi por completo de la configuración inicial y no del algoritmo en sí mismo.

Recocido simulado logra un resultado más equilibrado: es más costoso que Ascenso de colina, pero al aceptar temporalmente movimientos que empeoran el puntaje, es capaz de escapar de óptimos locales y encontrar mejores soluciones sin llegar al costo del algoritmo genético. En nuestras pruebas, Recocido simulado encontró el máximo global en el layout más pequeño, mientras que el algoritmo genético no lo logró; en cambio, en un layout con zonas de mejora y desmejora más claramente diferenciadas ocurrió lo contrario. Esto sugiere que el algoritmo genético obtiene mayor ventaja en mapas con regiones bien distinguidas entre configuraciones buenas y malas, donde la diversidad poblacional puede explotar esa estructura, mientras que Recocido simulado es más eficiente en espacios de búsqueda más simples o pequeños.

Algoritmo genético fue, de los tres, el más costoso computacionalmente: en los peores casos tardó hasta 15 veces más y realizó 22 veces más evaluaciones que Ascenso de colina. Esta inversión solo se justifica cuando el layout presenta suficiente estructura (zonas de mejora y desmejora bien diferenciadas) para que la exploración poblacional aporte una ventaja real sobre los otros métodos. En layouts pequeños o poco complejos, como los usados en este taller, dicho costo adicional no se traduce en una mejora proporcional de la solución, por lo que el algoritmo genético resulta una exageración frente al problema.

Esto muestra que el costo computacional no garantiza por sí solo una mejor calidad de solución: Ascenso de colina es barato pero inconsistente, Recocido simulado logra resultados casi equivalentes a un costo mucho menor cuando se ajustan bien sus parámetros, y Algoritmo genético solo justifica su alto costo en instancias con mayor complejidad estructural que las evaluadas en este taller.

d. Después de probar múltiples mapas, se lograron identificar diferentes casos en los que la profundidad influye directamente en el resultado de la partida. En particular, al utilizar el algoritmo de Minimax sobre el mapa `deceptive_distance` se identificaron resultados diferentes: cuando la profundidad es 1, el agente defensor no logra prever más allá de su propio movimiento y el agente intruso logra interceptarlo (resultado = derrota); cuando la profundidad es 2, cada agente logra prever un movimiento de su contrincante y ambos quedan en una esquina sin realizar movimientos hasta que se acaban las rondas (resultado = límite); y cuando la profundidad es 4, el agente defensor logra esquivar al agente intruso y activa todas las terminales críticas (resultado = victoria).

Esta diferencia entre resultados se debe principalmente a la interacción entre la función de evaluación implementada y el efecto horizonte que se da. Propiamente, cuando hay poca profundidad de plies o se llega al límite, el agente en turno queda "ciego" a lo que sucede justo después del límite de corte, en este caso queda ciego ante los otros movimientos que puede realizar su adversario (efecto horizonte); por esto, tiene que pasar a utilizar la función de evaluación, la cual le da una heurística que lo conduce hacia su meta, pero que es incapaz de evitar la toma de decisiones que pueden llevar al agente a un error inminente.

e. Explique por qué alfa-beta debe producir la misma decisión que Minimax bajo las mismas condiciones. Analice cómo la profundidad afecta el costo de la búsqueda y cómo el orden de las acciones afecta la eficacia de la poda alfa-beta.

Alfa-Beta y Minimax deben producir la misma decisión porque en el árbol de juego el turno MIN y el turno MAX se comportan de la misma manera. En el caso de MAX propagando su hijo mayor hacia arriba, y en el caso de MIN propagando su hijo menor hacia arriba. Este mecanismo produce la misma respuesta, pues la única diferencia es que Alfa-Beta realiza

una poda sobre aquellas ramas que ya no pueden cambiar el valor que MIN o MAX propagarán hacia arriba.

Por otra parte, la profundidad seleccionada para el algoritmo determina la expansión del árbol de juego. A mayor profundidad se realizan más turnos y se expanden más nodos antes de llegar a las hojas. Los aumentos en profundidad producen, por ende, tanto en minimax como alfaBeta, un crecimiento en el número de nodos visitados y el tiempo de ejecución.

Finalmente, el orden de las acciones afecta la eficacia de la poda Alfa-Beta, porque si se exploran primero acciones que permiten encontrar rápidamente valores altos para max y pequeños para min, haciendo que se actualicen antes los valores alfa y beta y se puedan realizar podas. Si el orden es desfavorable, los límites se actualizan más adelante y no se podan tantos elementos.

6b. Reflexión sobre el uso de IAG

Durante el taller, empleamos la inteligencia artificial como una herramienta de validación para nuestros algoritmos de optimización y juegos adversarios. Tras programar las versiones iniciales a partir de pseudocódigo, recurrimos a la IA para corregir errores, aclarar conceptos sobre los algoritmos y tener un mayor entendimiento sobre la estructura previamente implementada del taller. Esto no solo optimizó la eficiencia y el rendimiento de las soluciones, sino que fortaleció nuestro entendimiento sobre el propósito de cada algoritmo dentro del proyecto y su ejecución.

En el proceso aprendimos a formular mejores *prompts*: notamos que la calidad de las respuestas depende enteramente del contexto suministrado, lo que nos llevó a refinar nuestras preguntas en varias ocasiones. La IA fue una herramienta fundamental para la correcta implementación de los algoritmos, pues nos permitía contrastar implementaciones propias con los requisitos exigidos por la guía del taller.